

高级语言程序设计

实验报告

南开大学 计算机学院

姓名：闻鹏杰

学号：2511565

班级：1024

日期：2026 年 5 月

目录

一、作业题目与实验目的

二、开发环境

三、设计架构

四、主要功能与实现

五、技术难点与解决方案

六、总结与收获

七、代码仓库与讲解视频

一、 作业题目与实验目的

1. 作业题目

智能桌面角色

2. 实验目的

1. 综合运用 C++ 面向对象编程、Qt 框架及网络编程知识，开发一款具有交互性、实用性和趣味性的桌面宠物程序。
2. 实现角色切换、音乐播放、实时天气查询、待办事项管理、陪伴模式等核心功能。
3. 掌握 Qt 的信号槽机制、事件处理、布局管理、多媒体播放、网络请求与 JSON 解析等关键技术

二、 开发环境

操作系统: Windows 10

开发工具: Qt Creator

开发框架: Qt 6

构建工具: CMake

三、 设计架构

2.1 整体架构

程序采用 MVC 变体架构:

- 1、模型 (Model): 角色数据 (RoleData)、待办事项 (TodoItem)
- 2、视图 (View): DesktopPet (主窗口)、PetMenu (右侧菜单)、TodoDialog (添加待办对话框)
- 3、控制 (Controller): 信号槽连接、事件处理 (鼠标事件、定时器)
主窗口 DesktopPet 继承自 QWidget, 设置为无边框、透明、置顶的窗口。左侧信息面板显示时间、天气和待办列表; 右侧菜单 PetMenu 提供各种交互控件; 宠物区域显示当前角色的动画/静态图片, 并响应鼠标事件。

2.2 模块划分

- 1、角色管理模块: loadRoles()、setCurrentRole()、setPetImage()
- 2、音乐模块: QMediaPlayer + QAudioOutput, 支持循环播放
- 3、天气模块: QNetworkAccessManager 请求 OpenWeatherMap API, 解析 JSON
- 4、待办模块: TodoDialog 输入, checkTodos() 定时检查过期和临近截止
- 5、陪伴模式: 隐藏菜单/待办区域, 切换到 GIF 角色, 禁用角色切换
- 6、UI 自适应: updateWindowSize() 动态调整窗口大小

四、 主要功能与实现

1、角色切换与交互反馈

功能说明：用户可以在右侧菜单选择不同角色，每个角色拥有常态、单击、双击、长按四张图片和专属背景音乐。鼠标单击/双击/长按角色时，切换对应图片并显示气泡。

```
// 角色数据结构 (desktop_pet.h)
struct RoleData {
    QString normalImage;
    QString clickImage;
    QString doubleClickImage;
    QString longPressImage;
    QString musicPath;
    bool isGif;
};

// 鼠标释放事件 (识别单击)
void DesktopPet::mousePressEvent(QMouseEvent *event) {
    if (event->button() == Qt::LeftButton) {
        if (m_longPressTimer->isActive()) {
            m_longPressTimer->stop();
            setPetImage("click");
            showBubbleWithPattern("😊");
            m_resetTimer->start();
        }
        m_mousePressed = false;
        event->accept();
    }
}
```

2、智能去白底

功能说明：原始角色图片为白底，但宠物身上也有白色（如小白兔）。简单全局阈值会误删内部白色。采用**连通域填充算法**：从图片四边开始，将与背景色（白）相连的连续像素设为透明，从而只消除外部背景。

```

// ===== 去白底辅助函数 =====
// ===== 智能去除外部白色背景（保留内部白色） =====
QPixmap removeExternalWhite(const QPixmap &src, int tolerance = 30)
{
    if (src.isNull())
        return src;

    QImage img = src.toImage().convertToFormat(QImage::Format_ARGB32);
    int w = img.width();
    int h = img.height();
    if (w == 0 || h == 0)
        return src;

    // 取样背景色（取左上角像素颜色）
    QRgb bgColor = img.pixel(0, 0);

    // 标记数组，记录已处理的像素
    QVector<bool> visited(w * h, false);
    auto index = [w](int x, int y) { return y * w + x; };

    // 队列，用于BFS填充
    QVector<QPoint> queue;

    // 从四条边上的所有像素开始，如果颜色与背景色相近则入队
    auto shouldFill = [&](int x, int y) -> bool {
        if (x < 0 || x >= w || y < 0 || y >= h) return false;
        if (visited[index(x, y)]) return false;
        QRgb col = img.pixel(x, y);
        int dr = qAbs(qRed(col) - qRed(bgColor));
        int dg = qAbs(qGreen(col) - qGreen(bgColor));
        int db = qAbs(qBlue(col) - qBlue(bgColor));
        return (dr <= tolerance && dg <= tolerance && db <= tolerance);
    };

    // 上边和下边
    for (int x = 0; x < w; ++x) {
        if (shouldFill(x, 0)) {
            queue.append(QPoint(x, 0));
            visited[index(x, 0)] = true;
        }
        if (shouldFill(x, h-1)) {
            queue.append(QPoint(x, h-1));
            visited[index(x, h-1)] = true;
        }
    }

    // 左边和右边（避免重复角落，但重复也没关系）
    for (int y = 0; y < h; ++y) {
        if (shouldFill(0, y)) {
            queue.append(QPoint(0, y));
            visited[index(0, y)] = true;
        }
        if (shouldFill(w-1, y)) {
            queue.append(QPoint(w-1, y));
            visited[index(w-1, y)] = true;
        }
    }

    // 四个方向
    const int dx[] = {1, -1, 0, 0};
    const int dy[] = {0, 0, 1, -1};

    int head = 0;
    while (head < queue.size()) {
        QPoint p = queue[head++];
        int x = p.x(), y = p.y();
        // 将该像素设置为透明
        img.setPixel(x, y, qRgba(qRed(bgColor), qGreen(bgColor), qBlue(bgColor), 0));

        for (int i = 0; i < 4; ++i) {
            int nx = x + dx[i];
            int ny = y + dy[i];
            if (shouldFill(nx, ny)) {
                visited[index(nx, ny)] = true;
                queue.append(QPoint(nx, ny));
            }
        }
    }

    return QPixmap::fromImage(img);
}

```

3、天气查询

功能说明：使用 OpenWeatherMap API 获取实时天气（温度、湿度、天气描述）。支持用户输入任意城市名，点击“Go”按钮查询，并自动保存最近 5 个城市到 QSettings。

```
// ⚠ 请替换为你的 OpenWeatherMap API Key  
m_apiKey = "7b21b186-509b-409b-b01b-2a600181b06b1b";
```

```
void DesktopPet::requestWeatherForCity(const QString &cityName) {
    QUrl url("https://api.openweathermap.org/data/2.5/weather");
    QUrlQuery query;
    query.addQueryItem("q", cityName);
    query.addQueryItem("appid", m_apiKey);
    query.addQueryItem("units", "metric");
    query.addQueryItem("lang", "zh_cn");
    url.setQuery(query);
    m_weatherNetManager->get(QNetworkRequest(url));
}

void DesktopPet::onWeatherReplyFinished(QNetworkReply *reply) {
    // 解析 JSON, 更新 m_weatherLabel
}
```

4、待办事项管理

功能说明：用户填写待办名称和截止时间（精确到年月日时分）。待办列表自动编号，临近截止 24 小时内文字变红色，超时自动删除。支持显示/隐藏待办列表，一键清除所有待办。

```
void DesktopPet::checkTodos() {
    QDateTime now = QDateTime::currentDateTime();
    for (int i = m_todos.size() - 1; i >= 0; --i) {
        if (m_todos[i].dueTime <= now) { // 过期删除
            m_todoLayout->removeWidget(m_todos[i].displayLabel);
            delete m_todos[i].displayLabel;
            m_todos.removeAt(i);
        }
    }
    for (TodoItem &item : m_todos) {
        qint64 secsTo = now.secsTo(item.dueTime);
        if (secsTo <= 86400 && secsTo > 0) // 24小时内变红
            item.displayLabel->setStyleSheet("color: red;");
    }
}
```

5、陪伴模式

功能说明：点击右上角星形按钮，程序隐藏右侧菜单和待办区域，切换到第四个特殊角色（GIF 动图），进入沉浸式陪伴模式。再次点击退出，恢复原状。GIF 每一帧也会实时去白底。

```
void DesktopPet::toggleCompanionMode() {
    if (!m_companionMode) {
        m_companionMode = true;
        m_previousRoleIndex = m_currentRoleIndex;
        setCurrentRole(3);           // 切换到 GIF 角色
        m_menu->setRoleComboEnabled(false);
        m_menu->setVisible(false);
        m_todoGroup->setVisible(false);
        updateWindowSize();
    } else {
        // 退出逻辑...
    }
}
```

五、 技术难点与解决方案

无边框窗口拖拽	重写 mousePressEvent、mouseMoveEvent，计算偏移量并调用 move()
GIF 动图去白底	使用 QMovie，连接 frameChanged 信号，每帧调用 removeExternalWhite 再显示
窗口自适应收缩	调用 adjustSize() 并重新设置 fixedSize，先保存位置再恢复
中文字符及 emoji 编译	CMake 中添加/utf-8 编译选项，源文件保存为 UTF-8 with BOM
网络请求异步不阻塞	使用 QNetworkAccessManager 的 finished 信号

六、 总结与收获

本次大作业成功实现了一款功能丰富、交互友好的桌面宠物程序。通过实践，我深入理解了 Qt 的核心机制（信号槽、事件系统、布局管理、网络编程），掌握了 C++ 面向对象设计在 GUI 项目中的应用，提升了解决实际问题的能力。本作业部分代码使用 Deepseek 编写。

七、 代码仓库与讲解视频

Github 代码仓库: [andywen1103-png/-/NKU2026C-DesktopPet](https://github.com/andywen1103/png/-/NKU2026C-DesktopPet)

Bilibili 讲解视频: [【南开大学 26C++】在游戏之外，我也想遇到我喜欢的角色... 哔哩哔哩 bilibili](#)